

Lab 21 — On-Chip Debugging: JTAG, OpenOCD and GDB

ESP32-S3 FIRMWARE LAB SERIES · PHASE 3 · GAP-CLOSERS · GAP-CLOSER

Estimated time	2-3 hrs
Proves	breakpoints, watchpoints, RTOS-aware inspection
Prerequisites	Labs 0 and 7.
Beyond the board	Nothing extra — the JTAG port is in the silicon. Use the native USB port.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§21.1 objective**, **§21.2 parts and wiring**, **§21.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

KEY POINT

How this lab is structured, and why it is worth the setup

JTAG appears in firmware job postings constantly, and it is the clearest line between "I print things" and "I debug things." The ESP32-S3 has a JTAG debug port *built into the silicon*, reachable over the same USB cable, so it costs nothing extra — on most microcontrollers it needs a separate probe.

This lab supplies a **complete program containing a deliberate memory-corruption bug**, then walks you through catching it with a watchpoint. You are not debugging a hypothetical.

21.1 Objective & TM4C bridge

Stop debugging with `printf`. Halt the chip, inspect any variable, step through code, and catch a memory corruption with a watchpoint. This lab makes you actually do it, because a debugger you have configured once is a tool you will reach for forever, and one you have only read about is not.

TM4C BRIDGE

Keil gave you a debugger behind a button, backed by the ICDI chip on the LaunchPad. You may well have used breakpoints there without thinking about the machinery. Here you assemble it yourself from OpenOCD and GDB, which is how it works on nearly every professional embedded toolchain — and understanding the pieces means you can debug on hardware that has no polished IDE.

21.2 Parts & wiring

Item	Status	Notes
ESP32-S3 board	JTAG is inside the chip	Reachable over the port marked USB . The UART port cannot do this.
External JTAG probe	not needed	Do not buy a J-Link for this board. Those exist for chips without built-in JTAG. Save the money.

No wiring. One driver step on Windows: the JTAG interface needs the **WinUSB** driver bound with **Zadig** — the same tool and the same procedure you already used to get the logic analyzer working, so this should feel familiar.

21.3 How it works

Every lab so far has debugged with `printf`. That works, and it has three specific limitations worth naming before you replace it.

What is wrong with `printf` debugging

It changes what you are measuring. A serial write at 115200 baud takes roughly 87 μ s per character. Put one inside a 1 kHz sampling loop and you have consumed a meaningful slice of your 1000 μ s budget — the classic case where adding a print makes the bug disappear, or creates a new one. Anything timing-sensitive is distorted by the act of observing it.

You have to predict what you will want. Every `printf` is a guess made in advance about which variable matters. Guess wrong and you edit, rebuild, reflash and try again — a cycle of a minute or two, repeated dozens of times.

It cannot answer "who did this?" When a variable is being corrupted by something, no amount of printing tells you which line is responsible, because you would have to instrument every possible writer.

A debugger fixes all three: it halts the chip and lets you inspect *anything*, decided at the moment you want it rather than in advance.

The three cooperating programs

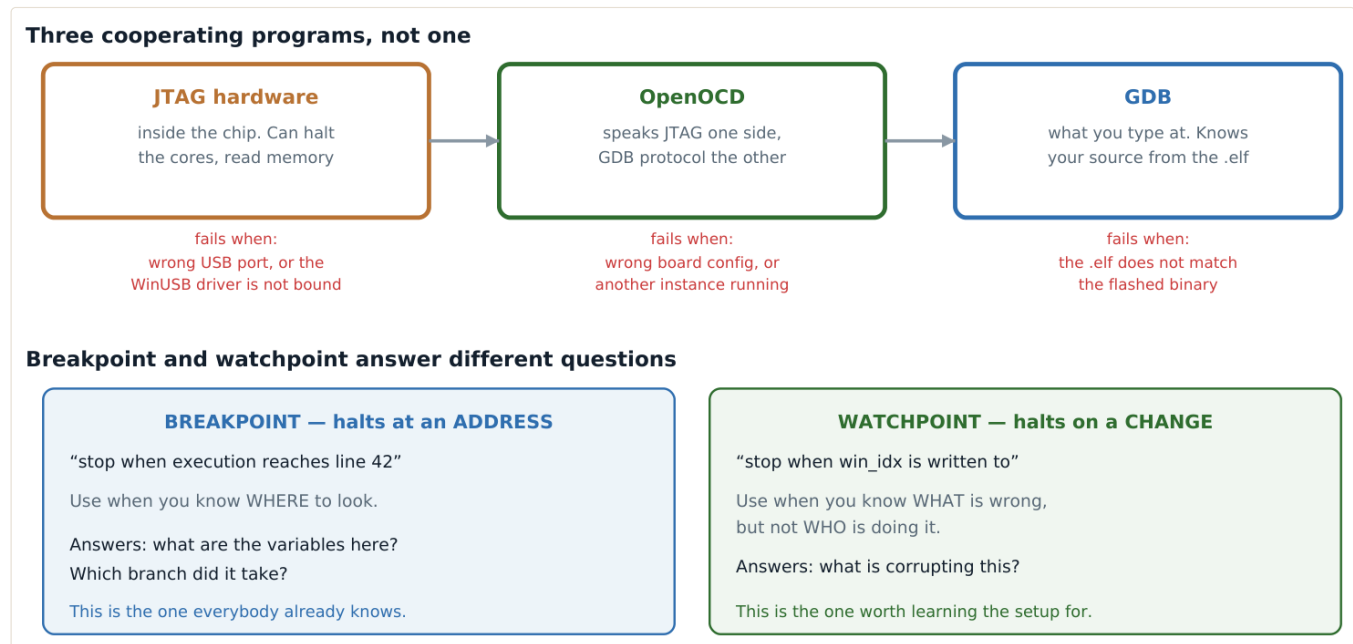


Figure 21.1. Top: debugging looks like one tool and is actually three programs cooperating — knowing which is which turns “it will not connect” into a diagnosis. Bottom: breakpoints and watchpoints answer genuinely different questions.

The three cooperating programs in a debug session, and breakpoints versus watchpoints.

Piece	What it is	Fails when
JTAG hardware	inside the chip; can halt the cores and read memory	wrong USB port, or the WinUSB driver is not bound
OpenOCD	speaks JTAG on one side, the GDB protocol on the other	wrong board config, or another instance is already running
GDB	what you type at; knows your source from the .elf	the .elf does not match the flashed binary

BREAKPOINT halts at an ADDRESS: “stop when execution reaches line 42”. Use it when you know WHERE to look. It answers: what are the variables here, and which branch did it take? This is the one everybody already knows.

WATCHPOINT halts on a CHANGE: “stop when win_idx is written to”. Use it when you know WHAT is wrong but not WHO is doing it. It answers: what is corrupting this? This is the one worth learning the setup for.

JTAG is hardware inside the silicon: a debug port that can halt the cores, read and write memory and registers, and set hardware breakpoints. It is an industry-standard interface, originally designed for testing manufactured boards. **The ESP32-S3 has it built in and reachable over the same USB cable**, which is why this lab costs nothing — on many microcontrollers you would need a separate probe.

OpenOCD is a translator. It speaks JTAG to the chip on one side and the GDB remote-serial protocol on the other, over a TCP socket. It is what makes a generic debugger able to drive chip-specific hardware.

GDB is what you actually type at. It knows nothing about JTAG; it knows your *source code*, because it reads the `.elf` file your build produced, which contains the symbol table and line-number mapping alongside the machine code.

WARNING

The `.elf` must match what is flashed

GDB maps addresses to source lines using the `.elf`. Flash one build, debug against a different `.elf`, and it will confidently show you the wrong lines — with no warning, because nothing checks. Always debug the build you just flashed, and keep the `.elf` from anything you deploy, because without it a crash log from the field is close to unreadable.

Breakpoints and watchpoints answer different questions

A breakpoint halts when execution reaches an address. "Stop at line 42." Use it when you know *where* to look, and want to know what the state is at that point or which branch was taken. This is the one everybody already knows.

A watchpoint halts when a memory location changes. "Stop when `win_idx` is written to." Use it when you know *what* is wrong but not *who* is doing it.

Watchpoints are the reason to bother with this setup. Memory corruption is the hardest class of embedded bug precisely because the crash happens somewhere unrelated to the cause — something writes past a buffer, and the program dies later in innocent code. A watchpoint on the corrupted address stops the chip *at the instruction responsible* and shows you the backtrace. That turns a multi-day bug into a ten-minute one.

Both are implemented by dedicated comparators in the silicon, so you get only a handful of each — typically two watchpoints on this chip. GDB will tell you when you run out. **Software breakpoints** work differently, by temporarily replacing an instruction with a trap, so you can have many more of those — but only in RAM-resident or writable code.

RTOS awareness, which matters more than it sounds

ESP-IDF's GDB integration understands FreeRTOS. When you halt, you can list *every* task and inspect the stack of any one of them, not just whichever happened to be executing.

For a deadlock that is the entire answer: the interesting question is not what the running task is doing, it is what every *blocked* task is waiting on. `info threads` lists them; `thread N` switches. Without RTOS awareness you would see one stack and nine mysteries.

What debugging does to timing, and when that matters

Halting stops your code but not the physical world. A motor keeps spinning. A communications peer times out. A watchdog may fire the moment you resume.

For pure logic bugs this is irrelevant. For timing-dependent bugs it can make the problem vanish — and **that vanishing is itself diagnostic**. If a bug disappears under the debugger, you have learned

something real: it is a race or a timing dependency, not a logic error, and the right tool is the logic analyzer rather than GDB.

Practical consequence: disable the task watchdog while debugging, or every breakpoint you hit will be followed by a reset.

Why optimisation must come down

At `-O2` the compiler reorders instructions, merges and eliminates variables, and inlines functions. The machine code no longer corresponds line-by-line to your source, so stepping jumps around apparently at random and variables report as **"optimized out"** — they genuinely do not exist as storage, having been kept in a register or computed away entirely.

`-Og` exists precisely for this: optimisation chosen to preserve debuggability. Set it in `menuconfig` → Compiler options before you start, and remember to put it back for anything you measure performance on.

21.4 Phase 0 — configure, cable, driver

0.1 — project files

```
// sdkconfig.defaults -- note the compiler setting, it matters here
CONFIG_IDF_TARGET="esp32s3"
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
CONFIG_COMPILER_OPTIMIZATION_DEBUG=y
CONFIG_ESP_TASK_WDT_INIT=n
CONFIG_ESP_SYSTEM_PANIC_GDBSTUB=y
```

Setting	Why it is here
<code>COMPILER_OPTIMIZATION_DEBUG</code>	Builds at <code>-Og</code> . At <code>-O2</code> the compiler reorders instructions, merges variables and inlines functions, so stepping jumps around apparently at random and variables report as "optimized out" — they genuinely do not exist as storage. <code>-Og</code> is optimisation chosen to preserve debuggability.
<code>ESP_TASK_WDT_INIT=n</code>	Halting at a breakpoint stops your code but not the watchdog. Leave it on and every breakpoint you hit is followed by a reset.
<code>PANIC_GDBSTUB</code>	On a crash the chip waits for a debugger instead of rebooting, so you can inspect the moment of death.

0.2 — the cable, which is the most common failure

WARNING

Use the port marked USB, not the one marked COM or UART

The board has two USB-C ports. The **UART bridge** port flashes and logs but has **no JTAG**. The **native USB** port carries the ESP32-S3's own USB-Serial-JTAG peripheral, and it is the only one that can debug.

If OpenOCD says it cannot find a device, check this before anything else. It is the answer more often than every other cause combined.

0.3 — Windows driver, once

Windows binds its own driver to the JTAG interface and OpenOCD cannot claim it. Rebind it with **Zadig**, the same utility used to bind WinUSB to a cheap logic analyzer.

1. Plug into the **native USB** port.
2. Open Zadig. **Options → List All Devices**.
3. In the dropdown choose **USB JTAG/serial debug unit (Interface 2)**. Interface 2 is the JTAG one — interface 0 is the serial port and you must not touch it.
4. Set the target driver to **WinUSB** and click **Replace Driver**.

Verify:

```
Get-PnpDevice -FriendlyName "*JTAG*" | Format-Table FriendlyName, Status
```

You want **OK**. Do this once per machine, not per project.

0.4 — confirm OpenOCD sees the chip

```
openocd -f board/esp32s3-builtin.cfg
```

KEY POINT

Expected — leave this window running

```
c Info : esp_usb_jtag: VID set to 0x303a and PID to 0x1001 Info : [esp32s3.cpu0] Examination
succeed Info : [esp32s3.cpu1] Examination succeed Info : starting gdb server for esp32s3.cpu0
on 3333
```

Two cores examined and a server on port **3333**. That is your connection proven, before GDB is involved at all.

WARNING**If it fails**

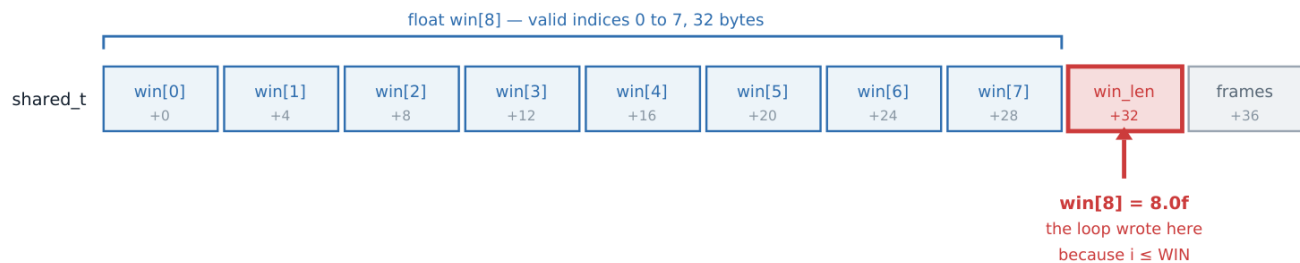
Message	Cause
no device found	Wrong USB port, or the WinUSB driver is not bound (§0.3)
LIBUSB_ERROR_ACCESS	Another OpenOCD is already running, or a serial monitor holds the port. Close both.
Examination failed	Board in download mode — press RESET and retry

21.5 Phase 1 — a program with a real bug in it

This writes one element past the end of an array. In C that is undefined behaviour, and here it corrupts the variable that sits next in memory — which the consumer then uses as a loop bound. **The crash happens in a different task, in code that is entirely innocent.** That is exactly the class of bug a watchpoint exists for.

Why one wrong character corrupts a different variable

The struct in Lab 21, laid out byte by byte. Offsets are from the start of the struct.



The array has 8 elements, so `win[8]` is one past the end — and one past the end IS `win_len`.

A float write lands on a `uint32_t`. The bytes are reinterpreted, not converted:

8.0f as IEEE-754 single precision = 0x41000000 = 1090519040 when read as an integer.

A huge value that decodes to a small float is a fingerprint: it means a float write landed on an integer.

And nothing warns you

C does not bounds-check. The compiler emits the store, the CPU performs it, and the program continues.

The crash arrives later, in the consumer, on a line that is entirely correct — which is why the debugger, not the panic message, is what finds it.

Figure 21.2. [DERIVED] The struct laid out byte by byte. `win[]` holds 8 floats and ends at offset 32, which is exactly where `win_len` begins. Writing `win[8]` writes `win_len`.

Byte layout of the Lab 21 struct, showing why one wrong character corrupts a different variable. [DERIVED]

`shared_t` laid out left to right with byte offsets:

Member	Offset	Note
<code>win[0]</code>	+0	
<code>win[1]</code>	+4	
<code>win[2]</code>	+8	
<code>win[3]</code>	+12	
<code>win[4]</code>	+16	
<code>win[5]</code>	+20	
<code>win[6]</code>	+24	
<code>win[7]</code>	+28	last valid element
<code>win_len</code>	+32	highlighted in red - this is where <code>win[8]</code> would be
<code>frames</code>	+36	

A brace spans offsets 0 to 31 labelled `float win[8]` - valid indices 0 to 7, 32 bytes. An arrow points up at the `win_len` cell labelled `win[8] = 8.0f`, the loop wrote here, because `i <= WIN`.

The array has 8 elements, so `win[8]` is one past the end - and one past the end IS `win_len`. They are the same address, and the compiler is not wrong to allow it.

A float write lands on a `uint32_t`. The bytes are reinterpreted, not converted: `8.0f` as IEEE-754 single precision is `0x41000000`, which is **1090519040** read as an integer. A huge value that decodes to a small float is a fingerprint - it means a float write landed on an integer.

And nothing warns you. C does not bounds-check. The compiler emits the store, the CPU performs it, and the program continues. The crash arrives later, in the consumer, on a line that is entirely correct - which is why the debugger, not the panic message, is what finds it.

The figure is the whole bug in one picture, and it is worth reading the offsets rather than taking the conclusion on trust. Eight floats at four bytes each occupy offsets 0 through 31. The next member starts at 32. And `win[8]` — the element that does not exist — would also start at 32. **They are the same address, and the compiler is not wrong to allow it.** C does not bounds-check.

Note the value, because it is a diagnostic fingerprint you will use again. The float `8.0f` stored as IEEE-754 is `0x41000000`, which read back as a `uint32_t` is **1090519040**. A suspiciously huge integer that decodes to a small, sensible float means *a float write landed on an integer*. Recognising that pattern in a register dump saves real time.

KEY POINT**Why this uses a struct and a `_Static_assert`**

An earlier edition declared `win[]` and `win_idx` as separate file-scope variables and relied on the compiler placing them adjacently. **Nothing in C guarantees that**, so the demo could silently do nothing on a different compiler or optimisation level — and a teaching example that might not reproduce is worthless.

Putting both in a struct makes the layout defined, and the `_Static_assert` *fails the build* if padding ever breaks the adjacency the demo depends on. The assertion is part of the lesson: when you rely on layout, assert it.

```
// =====
// Lab 21 -- a deliberate memory-corruption bug, caught with a watchpoint
//
// Goal:   Catch an out-of-bounds write at the instruction responsible, using
//         a hardware watchpoint, then inspect a blocked task's stack.
// Verify: The FIRST watchpoint hit lands on the array write, not on any line
//         that mentions the corrupted variable by name.
// Gotcha: Build at -Og. At -O2 variables report as "optimized out".
//         Use the NATIVE USB port -- the UART bridge port has no JTAG.
// =====
#include <stdio.h>
#include <stddef.h>
#include <string.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

static const char *TAG = "lab21";

#define WIN 8

// Layout is DEFINED here, not hoped for. win_len sits immediately after win[].
typedef struct {
    float    win[WIN];
    uint32_t win_len;      // the consumer's loop bound. The producer never
    uint32_t frames;       // writes this -- so any change to it is the bug.
} shared_t;

static shared_t g = { .win_len = WIN };

// If padding ever separates them, FAIL THE BUILD rather than silently
// producing a demo that does nothing.
_Static_assert(sizeof(shared_t) == sizeof(float) * WIN,
    "win_len must sit immediately after win[] for this lab to work");

static void producer_task(void *arg)
{
    (void)arg;
    uint32_t n = 0;
    for (;;) {
        // THE BUG: '<=' should be '<'. On the last pass i == WIN, so
        // g.win[8] is written -- which IS g.win_len. One character.
        for (int i = 0; i <= WIN; i++) {
            g.win[i] = (float)(n + i);
```

```

    }
    g.frames++;
    n++;
    vTaskDelay(pdMS_TO_TICKS(500));
}

static void consumer_task(void *arg)
{
    (void)arg;
    for (;;) {
        // Innocent code. It trusts win_len, which should always be 8.
        // Once corrupted it iterates about a billion times, walks off the
        // end of RAM, and faults -- in THIS task, not in the producer.
        float sum = 0.0f;
        for (uint32_t i = 0; i < g.win_len; i++) {
            sum += g.win[i];
        }
        ESP_LOGI(TAG, "len=%" PRIu32 " sum=%.1f frames=%" PRIu32,
                 g.win_len, (double)sum, g.frames);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void app_main(void)
{
    ESP_LOGW(TAG, "win=%p win_len=%p (adjacent by construction)",
             (void *)g.win, (void *)&g.win_len);
    xTaskCreate(producer_task, "prod", 4096, NULL, 5, NULL);
    xTaskCreate(consumer_task, "cons", 4096, NULL, 5, NULL);
}

```

```
idf.py build
idf.py -p COM6 flash monitor
```

KEY POINT

What you should see, within about a second

c W lab21: win=0x3fc98a20 win_len=0x3fc98a40 (adjacent by construction) I lab21: len=8 sum=28.0 frames=1 Guru Meditation Error: Core 0 panic'ed (LoadProhibited).

len=8 on the first pass, then the producer runs and the next consumer pass reads a corrupted length and dies. **Do not fix it by reading the code.** Find it with the debugger — that is the exercise.

[DERIVED] The corrupted value will be **1090519040**. That is the float `8.0f` reinterpreted as a `uint32_t`: IEEE-754 single precision encodes 8.0 as `0x41000000`, and `0x41000000` is 1090519040. Seeing a huge value that decodes to a small float is itself a strong hint that a float write landed on an integer.

21.6 Phase 2 — attach GDB

Three windows. OpenOCD from §0.4 stays running in the first. In a second:

```
idf.py gdb
```

That wrapper finds your `.elf` and connects to port 3333 for you. The manual equivalent, worth knowing:

```
xtensa-esp32s3-elf-gdb -ex "target remote :3333" build\lab21_debug.elf
```

WARNING

The `.elf` must match what is flashed

GDB maps addresses to source lines using the `.elf`. Debug against a different build and it will confidently show you the **wrong lines**, with no warning, because nothing checks. Always debug the build you just flashed.

21.7 Phase 3 — catch the bug with a watchpoint

Follow this exactly. It is the payoff for the whole setup.

WARNING

Set the watchpoint AFTER reaching `app_main`, not before

The obvious sequence — reset, set the watchpoint, continue — does not work, and the reason is worth understanding.

`g.win_len` is initialised to 8, so it lives in the `.data` section. During startup, before `app_main` runs, the C runtime copies `.data` from flash into RAM. **That copy writes `g.win_len`**, so a watchpoint armed at the reset vector fires immediately, inside startup code, pointing at something that is entirely correct.

Break at `app_main` first. By then `.data` is initialised and the next write really is your bug. (`idf.py gdb` generates a `gdbinit` that already stops at `app_main` for you — but knowing why it does is the point.)

```
(gdb) monitor reset halt
(gdb) break app_main
(gdb) continue                                # runs startup; .data now initialised

Breakpoint 1, app_main () at ../main/main.c:70

(gdb) delete 1                                # free the hardware slot, we are past it
(gdb) watch g.win_len                          # NOW arm it. Halt on any write.
(gdb) continue
```

KEY POINT**The first hit IS the bug — there is no innocent write to sit through**

This is the important structural point. `g.win_len` is written **exactly once** in the whole program, at initialisation. The producer never assigns it. So the first watchpoint hit after boot is the corruption itself:

```
```c Hardware watchpoint 2: g.win_len
```

```
Old value = 8 New value = 1090519040 producer_task (arg=0x0) at ../main/main.c:47 47 g.win[i] = (float)(n + i); ```
```

**Line 47 is the array write.** Nothing on that line mentions `win_len`. The debugger has named the responsible instruction for a variable it never textually touches — which is precisely what no amount of `printf` can do.

Now confirm the mechanism rather than assuming it:

```
(gdb) print i
$1 = 8 # the array has indices 0..7

(gdb) print WIN
$2 = 8

(gdb) print &g.win[8]
$3 = (float *) 0x3fc98a40

(gdb) print &g.win_len
$4 = (uint32_t *) 0x3fc98a40 # THE SAME ADDRESS. There is your bug.

(gdb) print/x g.win_len
$5 = 0x41000000 # = 8.0f as IEEE-754. A float landed here.

(gdb) backtrace
#0 producer_task (arg=0x0) at ../main/main.c:47
#1 0x40382b1c in vPortTaskWrapper at .../port.c:162
```

**KEY POINT****What just happened**

`&g.win[8]` and `&g.win_len` print the **same address**. Writing one element past the array wrote the length. `print/x` shows `0x41000000`, which is 8.0 in IEEE-754 — so you can tell not just *that* it was corrupted but *what kind of value* did it.

The fix is one character: `i <= WIN` becomes `i < WIN`.

**NOTE****The address in \$3 and \$4 will differ on your board**

`0x3fc98a40` is illustrative. What must match is that the two *are equal to each other* — that equality is the finding, not the particular number. Your `app_main` prints both at boot for the same reason.

### 21.7.1 — The same bug without a watchpoint, for contrast

Worth doing once so you have felt the difference. Comment out the watchpoint, let it crash, and look at the panic:

```
Guru Meditation Error: Core 0 panic'ed (LoadProhibited).
Core 0 register dump:
PC : 0x42008f1a ...
EXCVADDR: 0x40000000
Backtrace: 0x42008f1a:0x3fc9ab30 ...
#0 consumer_task at ../main/main.c:62
```

Line 62 is `sum += g.win[i];` — in the *consumer*, a task that contains no bug at all. The producer, which caused it, is not in the backtrace, because it is not on this stack and may not even have been running.

**That gap between where it dies and what broke it is the entire reason watchpoints exist.**

Chasing this from the panic alone means suspecting the consumer, then the buffer, then the RTOS, and eventually stumbling on the producer. The watchpoint went straight there.

## 21.8 Phase 4 — prove it

#	Do this	Expected
1	OpenOCD connects.	Both cores examined, server on 3333.
2	Breakpoint: <code>break app_main</code> then <code>continue</code> after <code>monitor reset halt</code> .	Halts at the top of <code>app_main</code> . Guaranteed reachable, so it isolates connection problems from breakpoint problems.
3	Watchpoint as in §21.7.	Second hit lands on <b>line 28</b> , the array write.
4	<code>info threads</code> while halted.	Every FreeRTOS task listed with its state — <code>prod</code> , <code>cons</code> , <code>IDLE</code> , <code>ipc</code> and others.
5	<code>thread 3</code> then <code>backtrace</code> .	The stack of a task that is <b>not running</b> . No <code>printf</code> equivalent exists for this.
6	Fix the <code>&lt;=</code> , reflash, rerun the watchpoint.	Only the legitimate write on line 31 ever fires.

#### KEY POINT

##### Case 5 is the one people do not expect

ESP-IDF's GDB integration is **RTOS-aware**. When you halt, you can inspect the stack of any task, not just the one that happened to be executing. For a deadlock that is the whole answer: the interesting question is not what the running task is doing, it is what every *blocked* task is waiting on.

## 21.9 The commands, one at a time

About fifteen commands cover everything. Abbreviations in brackets are what you will actually type.

Command	Does	Use when
<code>break f /</code> <code>break file.c:42 (b)</code>	Halt at an address	You know <b>where</b> to look
<code>watch var</code>	Halt when memory is <b>written</b>	You know <b>what</b> is wrong but not <b>who</b> did it
<code>rwatch / awatch</code>	Halt on read / on either	"Who is consuming this value?"
<code>continue (c)</code>	Resume until the next stop	Always
<code>next (n)</code>	One line, <b>stepping over</b> calls	Moving through code you trust
<code>step (s)</code>	One line, <b>stepping into</b> calls	Descending into the function you suspect
<code>finish</code>	Run until the current function returns	You stepped somewhere uninteresting; also prints the return value
<code>print expr (p)</code>	Evaluate anything in scope	The command that replaces <code>printf</code>
<code>print arr[0]@10</code>	<b>Artificial array</b> — 10 consecutive elements	Inspecting a whole buffer at once
<code>print/x</code>	Format: <code>/x</code> hex, <code>/t</code> binary, <code>/f</code> float, <code>/c</code> char	<code>/t</code> is excellent for checking register bit fields
<code>set var x = 0</code>	<b>Change a variable</b> , then continue	Forcing a branch you cannot easily reach
<code>backtrace (bt)</code>	The call chain that led here	After any halt — usually more informative than where you are
<code>info locals / info args</code>	Every local / every parameter	Faster than printing individually
<code>info threads</code>	Every FreeRTOS task and its state	Diagnosing a deadlock
<code>thread N</code>	Switch to another task's stack	Inspecting a task that is not running
<code>info watchpoints</code>	What you are watching	When a new watchpoint silently fails — you ran out
<code>delete N / disable N</code>	Remove / suspend	Freeing a hardware slot

**NOTE****Hardware limits, and why watchpoints run out**

Breakpoints and watchpoints are implemented by dedicated comparators in the silicon, so you get only a handful — typically **two watchpoints** on this chip. GDB tells you when you exceed it. A watchpoint on a *local* variable also stops being meaningful once its function returns and the stack slot is reused.

**Software breakpoints** work differently, by temporarily replacing an instruction with a trap, so you can have many more — but only in writable memory.

## 21.10 Extensions, worked

### 21.10.1 — Forcing a branch you cannot otherwise reach

`set var` surprises people who think a debugger is read-only. It is how you test an error path without provoking the error.

```
(gdb) break consumer_task
(gdb) continue
(gdb) set var win_idx = 99 # force an out-of-range index
(gdb) continue # now watch what your code does with it
```

You just tested the corrupted-index path deliberately, in seconds, without waiting for the bug. In real work this is how you exercise a failure branch: force an error code, skip a wait, drive a state machine into the state you want. **You can prove a fix works before you write it.**

### 21.10.2 — When the bug disappears under the debugger

Halting stops your code but not the physical world. A motor keeps turning, a peer times out. So a timing-dependent bug can vanish when you attach.

#### KEY POINT

#### That vanishing is itself a diagnosis

If a bug disappears under the debugger, you have learned something real: **it is a race or a timing dependency, not a logic error.** Stop using GDB and reach for the logic analyzer and GPIO instrumentation from §7.7 instead. Knowing which tool a symptom calls for is most of debugging.

### 21.10.3 — Reading a crash without a debugger attached

With `CONFIG_ESP_SYSTEM_PANIC_GDBSTUB` the chip waits for GDB on a crash. But in the field you get only a log, and you can still decode it. Delete the `<=` fix, let it crash, and copy the backtrace line:

```
xtensa-esp32s3-elf-addr2line -pfiaC -e build\lab21_debug.elf 0x42014b22 0x420098ab
```

That prints function names and source lines from raw addresses. `idf.py monitor` does it automatically when it is attached, which is why crash dumps look decoded in the terminal — but a log emailed from the field is raw, and this is how you read it.

**Keep the `.elf` for every build you ship.** Without it, a field crash log is close to unreadable.

## 21.11 Docs and references

Topic	Where
JTAG debugging setup, all platforms	ESP-IDF v5.5 → API Guides → JTAG Debugging
Built-in USB-JTAG specifics	Same section → Configure ESP32-S3 built-in JTAG
Panic handler options	ESP-IDF v5.5 → API Guides → Fatal Errors

Topic	Where
GDB itself	<code>help</code> at the prompt, or the GNU GDB manual — the "Setting Watchpoints" chapter is worth twenty minutes

# Bench realities

*Problems that stop this lab but are not about the peripheral it teaches. Each one cost real time on a real bench.*

## Which USB port for which job

Task	Port	Why
Flashing and normal monitoring	Either	Both enumerate a serial port
<b>Unity interactive menu</b>	<b>UART bridge port</b> (COM6 on this bench)	Unity reads with a ROM-level <code>uart_rx_one_char</code> , which is bound to UART0, not the native USB device
<b>JTAG debugging (Lab 21)</b>	<b>Native USB port</b>	The USB-Serial-JTAG peripheral is only on that port

If a lab needs you to *type* at the device and nothing you type registers, you are almost certainly on the wrong port.

# The series at a glance

*Each lab is a complete, independently buildable ESP-IDF project. This is Lab 21.*

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I <sup>2</sup> C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
<b>21 ←</b>	<b>On-chip debugging: JTAG, OpenOCD, GDB</b>	<b>Gap-closers</b>
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

*Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.*